

EOAT Command Center Feature Expansion Implementation Plan

Purpose

This document defines the next major usability and workflow expansion for the `Kgray44/EOAT_Tool` repository.

The app already has a strong foundation: a PySide dashboard, EOAT audit capture, compatibility rows, notes/tags, open items, workbook validation, reports, scheduled reports, performance diagnostics, backup tools, release readiness tools, and final handoff workflows.

The goal of this implementation is **not** to randomly add new unrelated pages. The goal is to expand existing tools so the app becomes easier, faster, safer, and more useful during real EOAT audit work.

The app should evolve from:

“A dashboard with many tools”

into:

“A guided EOAT engineering workflow system that tells the user what to do next, what data is trustworthy, what still needs work, and what each action will change.”

This document is intentionally detailed. Implement in phases. Do not attempt the entire document in one giant commit.

1. Core Product Direction

1.1 Main Product Goal

Improve the app’s usability by expanding existing features into deeper workflows:

- Command Palette becomes the main action launcher.
- Audit page becomes a guided audit assistant.
- Audit Coach becomes a multi-profile completion engine.
- Press View becomes a machine intelligence dashboard.
- Open Items becomes a real work queue.
- Photos become managed visual evidence.
- FMEA becomes evidence-driven.
- Pilot scoring becomes transparent and defensible.

- KPI dashboard separates actual, estimated, and missing data.
- PM checklist generation becomes EOAT-type-specific.
- BOM/spares analysis becomes a standardization engine.
- Performance page explains why the app is slow.
- Release Readiness proves the repo is safe to publish.
- Architecture evolves toward a feature registry.

1.2 Non-Goals

Do not:

- Add cloud dependencies.
- Add internet-required runtime behavior.
- Commit real project data.
- Rewrite the whole app.
- Replace the existing page registry without a migration path.
- Remove current functionality.
- Build fake metrics.
- Hide uncertainty.
- Treat compatibility-derived data as physically verified data.
- Pretend estimated KPI data is measured KPI data.
- Make scheduled reports overwrite existing reports silently.
- Add decorative UI without workflow value.

1.3 Design Principles

Local-first

The app must continue working from local files and local project roots.

Truthful data state

The app must clearly distinguish:

- User-entered physical audit data
- Smart defaults
- Compatibility-derived data
- Robot info workbook data
- Unknown / Not Checked
- N/A because not applicable
- Missing
- Estimated
- Measured

Guided workflows

The app should actively help the user answer:

- What should I do next?
- What is safe to skip?
- What still blocks completion?
- What did the app autofill?
- What will saving change?
- What data is trustworthy?
- What should be photographed?
- What report should I generate?

Modular implementation

Keep business logic in `core/`.

Keep UI orchestration in `app/pages/`.

Keep reusable UI widgets in `app/widgets/`.

Keep command/search/feature metadata centralized.

Testable behavior

Every major workflow expansion should have regression tests.

2. Current Repo Areas Relevant to This Plan

The following files and modules are central to the implementation:

Dashboard and navigation

- `app/page_registry.py`
- `app/navigation.py`
- `app/dashboard_ui.py`
- `app/command_registry.py`
- `app/widgets/command_palette.py`
- `app/event_bus.py`
- `app/page_async.py`
- `app/task_runner.py`

Audit workflow

- `app/pages/audit.py`
- `app/pages/audit_coach_panel.py`
- `app/pages/audit_defaults_controller.py`
- `app/pages/audit_save_workflow.py`
- `app/pages/audit_compatibility_panel.py`
- `core/audit/coach.py`
- `core/audit_field_rules.py`
- `core/audit_entries.py`
- `core/audit_compatibility.py`
- `core/audit/drafts.py`
- `core/gripper_fields.py`
- `core/gripper_presets.py`
- `core/robot_info.py`

Press View

- `app/pages/press_view.py`
- `core/press_view.py`

Open Items

- `app/pages/open_items.py`
- `core/open_items.py`
- `core/action_items.py`

Search

- `core/search.py`
- `app/widgets/command_palette.py`

Notes, tags, annotations

- `app/pages/notes.py`
- `app/pages/tags.py`
- `core/annotations/service.py`
- `core/annotations/database.py`
- `core/annotations/migrations.py`
- `core/annotations/suggestions.py`
- `core/annotations/targets.py`
- `core/annotations/tag_colors.py`

Workbook health

- `app/pages/workbook_health.py`
- `core/validation.py`
- `core/validation_findings.py`
- `core/workbook_repairs.py`
- `core/audit_by_press.py`

Reports and scheduling

- `app/pages/reports.py`
- `app/pages/scheduled_reports.py`
- `core/reports.py`
- `core/scheduled_reports.py`
- `scripts/install_summary_schedules.ps1`
- `scripts/run_daily_summary.ps1`
- `scripts/run_weekly_summary.ps1`
- `scripts/check_summary_schedules.ps1`
- `scripts/uninstall_summary_schedules.ps1`

Photos

- `app/pages/photos.py`
- `core/photo_evidence.py`
- Existing `Photo Index` workbook sheet handling

Standards, KPI, FMEA, pilot candidates, PM, BOM

- `app/pages/standards_docs.py`
- `app/pages/kpi_dashboard.py`
- `app/pages/fmea.py`
- `app/pages/pilot_candidates.py`
- `app/pages/pm_checklists.py`
- `app/pages/bom_spares.py`
- related `core/` modules

Performance

- `app/pages/performance.py`
- `core/performance.py`

Safety and release

- `app/pages/release_readiness.py`
- `app/pages/backup_manager.py`

- `scripts/repo_safety_audit.py`
 - `scripts/pre_commit_check.ps1`
 - `scripts/install_pre_commit_hook.py`
-

3. Phase 0: Safety, Test, and Architecture Preparation

3.1 Objective

Before adding new workflows, make sure the existing architecture can accept expansion safely.

3.2 Required Work

3.2.1 Add or update developer notes

Create or update:

- `docs/feature_expansion_plan.md`
- `docs/architecture_notes.md`
- `docs/testing_strategy.md`

These documents should explain:

- How pages register.
- How commands register.
- How page events work.
- How background tasks work.
- How workbook locks should be used.
- How generated files should be safely written.
- How project-root data is kept outside the repo.

3.2.2 Create baseline tests

Add baseline tests before modifying behavior:

- Page registry loads every page spec.
- Navigation sections match page specs.
- Command palette opens with existing commands.
- Existing command registry has no duplicate command IDs.
- Audit field applicability still works for Vacuum, Mechanical / Gripper, Hybrid, Miscellaneous, and Unknown.
- Existing audit save tests still pass.
- Workbook Health validation still returns findings.
- Press View grouping still separates physical and compatible rows.

- Open Items still refuses manual “resolved” on generated items unless fixed at source or dismissed with reason.
- Scheduled Reports preview still detects expected daily and weekly dates.
- Repo safety audit still passes for staged/demo data.

3.3 Acceptance Criteria

- `python -m pytest` passes.
 - Existing UI smoke tests pass.
 - No real data is added.
 - No private paths are committed.
 - No generated reports/logs/cache/backups are committed.
 - Existing app still opens with demo project.
-

4. Phase 1: Expand Command Palette into the App Action Hub

4.1 Objective

The Command Palette should become the fastest way to navigate and run workflows.

Currently, the app has many pages, but the command registry only exposes a limited set of actions. Expand it so each major page can contribute its own commands.

4.2 New Concepts

4.2.1 Command provider interface

Add a reusable command provider pattern.

Suggested file:

- `app/command_provider.py`

Suggested classes:

```
from dataclasses import dataclass
from typing import Protocol

class CommandProvider(Protocol):
```

```
def commands(self, context) -> list[CommandSpec]:  
    ...
```

Or keep it simple:

```
def page_commands(page, window, config) -> list[CommandSpec]:  
    ...
```

4.2.2 Command context

Create a lightweight context object:

```
@dataclass(frozen=True)  
class CommandContext:  
    window: object  
    config: object  
    current_page_key: str | None  
    selected_audit_id: str = ""  
    selected_machine: str = ""  
    selected_field: str = ""
```

4.2.3 Disabled command reasons

Extend `CommandSpec`:

```
@dataclass(frozen=True)  
class CommandSpec:  
    command_id: str  
    display_name: str  
    aliases: tuple[str, ...] = ()  
    category: str = "Navigation"  
    handler: CommandHandler | None = None  
    safety_level: str = "safe"  
    requires_confirmation: bool = False  
    enabled: bool = True  
    disabled_reason: str = ""  
    description: str = ""
```

The UI should display disabled commands in gray and show the reason.

4.2.4 Context-aware ranking

Commands should be sorted by:

1. Enabled commands first
2. Commands from the current page
3. Recently used commands
4. Category
5. Display name

4.3 Commands to Add

4.3.1 Audit commands

Add commands:

- `audit.save_current`
- `audit.save_draft`
- `audit.discard_draft`
- `audit.clear_form`
- `audit.generate_new_id`
- `audit.duplicate`
- `audit.load_existing`
- `audit.jump_next_missing`
- `audit.mark_current_unknown`
- `audit.create_followup_current_field`
- `audit.review_suggestions`
- `audit.open_master_workbook`
- `audit.open_compatibility_tab`
- `audit.refresh_compatibility_sources`
- `audit.create_compatibility_entries`
- `audit.toggle_empty_only`
- `audit.toggle_full_audit`

4.3.2 Press View commands

Add commands:

- `press_view.refresh`
- `press_view.open_selected_audit`
- `press_view.open_selected_machine_detail`
- `press_view.export_selected_summary`
- `press_view.filter_open_items`
- `press_view.filter_pilot_candidates`
- `press_view.filter_validation_warnings`
- `press_view.filter_missing_photos`
- `press_view.copy_selected_machine_summary`

4.3.3 Open Items commands

Add commands:

- `open_items.refresh`
- `open_items.open_selected_source`
- `open_items.jump_to_target`
- `open_items.dismiss_selected`
- `open_items.create_note_from_selected`
- `open_items.export_report`
- `open_items.filter_overdue`
- `open_items.filter_critical`
- `open_items.filter_blocked`

4.3.4 Workbook Health commands

Add commands:

- `workbook_health.run_validation`
- `workbook_health.repair_schema`
- `workbook_health.refresh_audit_by_press`
- `workbook_health.preview_selected_fix`
- `workbook_health.apply_selected_fix`
- `workbook_health.create_annotation`
- `workbook_health.create_followup`
- `workbook_health.jump_to_field`
- `workbook_health.open_reports_folder`

4.3.5 Scheduled Reports commands

Add commands:

- `scheduled_reports.refresh_status`
- `scheduled_reports.run_preflight`
- `scheduled_reports.generate_daily_now`
- `scheduled_reports.generate_weekly_now`
- `scheduled_reports.run_daily_dry_run`
- `scheduled_reports.run_weekly_dry_run`
- `scheduled_reports.install_or_repair`
- `scheduled_reports.uninstall`
- `scheduled_reports.open_log`
- `scheduled_reports.open_latest_report`
- `scheduled_reports.catch_up_all_missed`

4.3.6 Photos commands

Add commands:

- `photos.open_incoming_folder`
- `photos.open_indexed_folder`
- `photos.intake_selected`
- `photos.refresh_index`
- `photos.find_missing_for_current_audit`
- `photos.generate_evidence_checklist`

4.3.7 Reports commands

Add commands:

- `reports.open_latest`
- `reports.open_reports_folder`
- `reports.generate_daily`
- `reports.generate_weekly`
- `reports.generate_kpi`
- `reports.generate_fmea`
- `reports.generate_documentation_gap`
- `reports.generate_handoff_package`

4.3.8 Release and safety commands

Add commands:

- `release.run_staged_safety_audit`
- `release.run_full_safety_audit`
- `release.copy_commit_checklist`
- `release.open_sanitization_report`
- `backup.preview_cleanup`
- `backup.open_backup_folder`

4.4 UI Changes

Update `app/widgets/command_palette.py`:

- Add disabled reason column or tooltip.
- Add recent commands.
- Add current page section.
- Add "Copy command ID" option.
- Add keyboard shortcuts:
 - Enter: run selected command
 - Ctrl+Enter: run without closing if safe
 - Esc: close

- Show command source page if available.
- Add “writes files” badge for commands where `modifies_files=True`.

4.5 Tests

Add tests for:

- No duplicate command IDs.
- All page-provided commands load.
- Disabled commands cannot execute.
- Disabled reason appears in command metadata.
- Context page commands rank above unrelated commands.
- Commands requiring confirmation do not execute if declined.
- Search still works while command list is populated.

4.6 Acceptance Criteria

- Ctrl+K opens a useful command palette.
- Every major page exposes at least 3 meaningful commands.
- Commands are context-aware.
- Unsafe commands require confirmation.
- Disabled commands explain why they are disabled.
- Existing commands still work.

5. Phase 2: Add Guided Audit Mode

5.1 Objective

The Audit page should guide the user through a physical EOAT audit step-by-step instead of just presenting a large form.

5.2 Current Strengths to Preserve

The Audit page already has:

- Audit Entry tab
- Compatibility Entry tab
- Interview Notes tab
- structured audit sections
- grouped field sections
- audit field visibility rules
- Empty Only / Full Audit mode
- machine lookup
- audit draft recovery

- Audit Coach
- field tags
- annotation suggestions
- background save
- compatibility sync preview
- robot info side effects
- workbook locking

Do not remove these.

5.3 New Guided Mode UI

Add a toggle near the top of the Audit Entry tab:

```
View Mode:
[ Guided Audit ] [ Section Form ]
```

The default can remain the current section form for now, but Guided Audit should be available.

5.4 Guided Audit Steps

Step 1: Identify machine and robot

Fields:

- Audit ID
- Audit Date
- Auditor
- Plant/Area
- Press/Machine #
- Robot Type
- Robot Model/Controller
- Tool #
- Part Family
- Part Name/Description

Add a lookup confidence card:

```
Machine Lookup:
Status: Exact match / Multiple matches / No match / Conflict
Matched source: capacity workbook / existing audit / manual entry
Warnings:
- Existing audit found for this machine
```

- Multiple part records found
- Robot model differs from previous audit

Step 2: Classify EOAT

Fields:

- EOAT Type
- EOAT Moves
- Connection Type
- Number of Parts Picked
- Estimated EOAT Weight

When EOAT type changes, show an applicability preview:

Visible because EOAT Type = Hybrid:

- Vacuum fields
- Gripper fields
- Sensor fields
- Pneumatic circuits

Hidden / N/A:

- none

Potential stale values:

- Cup Type still populated even if EOAT type changed to Mechanical

Step 3: Tooling details

Subcards:

Vacuum tooling

Fields:

• of Cups

- Cup Type/Material
- Cup Diameter/Size
- Vacuum Generator Type

Gripper tooling

Fields:

' of Grippers

- Gripper Type
- Gripper Model
- Gripper Size

Cylinder tooling

Add once cylinder fields are implemented:

' of Cylinders

- Cylinder Type
- Linear
- Rotary

Cylinder section should always be visible, but if both cylinder fields are blank, they should not count against completion.

Pickup summary

Show:

Parts picked per cycle:
Cups:
Grippers:
Cylinders:
EOAT moves:

Step 4: Pneumatic circuits

Fields:

- EOAT Vacuum Circuits
- EOAT Pressure Circuits
- EOAT Interchangeable Circuits
- Robot Vacuum Circuits
- Robot Pressure Circuits
- Robot Interchangeable Circuits

Add circuit confidence:

- Verified
- Estimated
- Unknown / Not Checked
- Derived from Robot_Info.xlsx

Add note:

EOAT-side circuit counts save to the EOAT Master Tracker.
Robot-side circuit counts save to Robot_Info.xlsx.

Step 5: Sensors and electrical

Fields:

- Sensors Present?
- Sensor Type
- Sensor Brand/Model
- Vacuum Confirmation Present?
- Part-Present Detection Present?
- Electrical/Wiring Present?
- Electrical Quick Disconnect Type
- Cable Management Condition

Show smart-default indicators:

Autofilled:
Sensor Type = Reed Switch
Sensor Brand/Model = SMC

Reason:
Part-Present Detection Present? is Yes.

Each autofilled value should have a visible badge:

- Defaulted
- User edited
- Verified
- Unknown

Step 6: Routing, mechanical, and reliability

Fields:

- Quick Disconnects Present?
- Pneumatic Quick Disconnect Type
- Tubing Condition
- Tubing Routing Notes
- Mounting Hardware Condition
- EOAT Alignment Condition
- Fastener/Locking Hardware Present?
- Known Issues
- Drop/Mis-Pick History
- Maintenance Frequency
- Cycle Time Concern?
- Scrap/Quality Concern?
- Changeover Difficulty

Add prompts:

- "Inspect tubing for pinch points, heat exposure, and excessive motion."
- "Check EOAT mounting hardware and alignment."
- "Record any drop/mis-pick history if known."
- "If not known, use Unknown / Not Checked instead of leaving ambiguous blanks."

Step 7: Documentation and photo evidence

Fields:

- Spare Parts Identified?
- Drawing/CAD Available?
- BOM Available?
- Process Binder Complete?
- Photos Taken?
- Photo Folder/Link

Add required shot checklist:

- Overall EOAT
- Tool connection
- Vacuum cups
- Grippers
- Cylinders
- Sensor mounting
- Tubing routing
- Quick disconnects
- Cable management
- Robot-side pneumatics

- Wear/damage
- Tool label or ID plate if available

For each shot:

- Required?
- Present?
- Linked?
- Missing?
- Not applicable?

Step 8: Final review and save impact

Before saving, show a pre-save summary:

- New audit or update existing audit
- Audit ID
- Machine
- Tool
- EOAT type
- fields changed
- fields defaulted
- fields cleared to N/A
- fields marked Unknown / Not Checked
- follow-up action will be created?
- robot info will be updated?
- compatibility rows will be updated?
- photos missing?
- validation warnings likely?

5.5 Pre-Save Diff

Add a reusable diff engine.

Suggested file:

- `core/audit/diff.py`

Suggested model:

```
@dataclass(frozen=True)
class AuditFieldChange:
    field: str
    old_value: str
    new_value: str
    change_type: str
    source: str
```

```

        warning: str = ""

@dataclass(frozen=True)
class AuditSavePreview:
    audit_id: str
    mode: str
    changes: list[AuditFieldChange]
    fields_to_na: list[str]
    smart_defaults: list[str]
    robot_info_updates: list[str]
    compatibility_impacts: list[str]
    warnings: list[str]

```

Change types:

- added
- changed
- cleared
- set_to_na
- smart_defaulted
- unknown_not_checked
- unchanged

Add UI:

- "Preview Save"
- "Save Anyway"
- "Cancel"
- "Copy Preview"

5.6 Tests

Add tests for:

- Guided step definitions include existing fields.
- Field applicability preview matches `field_applies`.
- Save preview detects changed fields.
- Save preview detects fields cleared to N/A.
- Save preview detects robot-side pneumatic changes.
- Save preview detects compatibility sync impacts.
- Cylinder fields do not count against completion when both blank.
- Autofilled sensor fields are marked as defaulted.

5.7 Acceptance Criteria

- User can complete an audit in Guided Audit mode.
- Existing Section Form mode still works.

- Save Preview clearly shows what will change.
 - Audit Coach stays in sync.
 - No hidden non-applicable stale values are silently kept.
 - Draft recovery still works.
 - Compatibility sync still works.
-

6. Phase 3: Upgrade Audit Coach into a Completion Policy Engine

6.1 Objective

Audit Coach should show multiple completion profiles instead of one generic percent complete.

6.2 Completion Profiles

Create profiles:

Quick Capture

Minimum fields needed to create a useful row.

Required:

- Audit Date
- Auditor
- Plant/Area
- Press/Machine #
- Robot Type
- EOAT Type
- Status

Recommended:

- Tool #
- Number of Parts Picked
- Photos Taken?

Physical Audit

Fields required to say a physical audit is complete.

Includes:

- machine/robot identity

- EOAT classification
- tooling details
- circuit details
- sensors
- quick disconnects
- tubing
- mechanical condition
- known issues
- photos

Engineering Analysis

Fields needed for FMEA, pilot ranking, and standards analysis.

Includes:

- Known Issues
- Drop/Mis-Pick History
- Maintenance Frequency
- Cycle Time Concern?
- Scrap/Quality Concern?
- Changeover Difficulty
- Priority
- Pilot Candidate?
- tags/notes/open items

Final Handoff Ready

Fields needed for project handoff.

Includes:

- Drawing/CAD Available?
- BOM Available?
- Process Binder Complete?
- Spare Parts Identified?
- PM checklist generated
- photo evidence complete
- no blocker validation findings

6.3 Data Model

Suggested file:

- `core/audit/completion_profiles.py`

Suggested model:

```

@dataclass(frozen=True)
class CompletionProfile:
    id: str
    label: str
    description: str
    required_fields: tuple[str, ...]
    important_fields: tuple[str, ...]
    optional_fields: tuple[str, ...]
    excluded_fields: tuple[str, ...] = ()

@dataclass(frozen=True)
class CompletionProfileResult:
    profile_id: str
    label: str
    percent_complete: int
    missing_required: tuple[str, ...]
    missing_important: tuple[str, ...]
    can_finish: bool
    next_best_field: str
    warnings: tuple[str, ...]

```

6.4 UI

Audit Coach should show:

```

Completion Profiles:
Quick Capture: 100%
Physical Audit: 86%
Engineering Analysis: 62%
Final Handoff Ready: 35%

```

Clicking a profile filters the guided field list to what matters for that profile.

6.5 Important Completion Rules

- Hidden non-applicable fields do not count.
- Fields explicitly marked Unknown / Not Checked count as known state but not verified complete.
- Fields with follow-up needed count as incomplete.
- Fields with stale hidden conflict count as incomplete.
- Cylinder fields are special:
 - If # of Cylinders and Cylinder Type are both blank, exclude from completion.
 - If either cylinder field is populated, both become part of completion.
- Tubing Routing Notes and final Notes should not count against completion unless marked required by a selected profile.

- Compatibility rows should not use the same completion rules as physical audits.

6.6 Tests

Add tests for:

- Each profile calculates independently.
- Unknown / Not Checked is not verified complete.
- N/A non-applicable fields are excluded.
- Stale hidden values block completion.
- Cylinder blank pair excluded.
- Cylinder partial entry becomes incomplete.
- Notes fields excluded by default.
- Physical audit and compatibility row completion differ.

6.7 Acceptance Criteria

- Audit Coach shows all completion profiles.
 - Current single percent complete can remain, but profile percentages must be visible.
 - User can select a profile and get next-best field for that profile.
 - Completion logic is tested independently from UI.
-

7. Phase 4: Expand Press View into Machine Intelligence

7.1 Objective

Press View should become the main machine-centered dashboard.

Current Press View groups physical audits, compatible entries, open items, validation warnings, photos, compliance, and pilot status. Expand it into a press/machine detail workflow.

7.2 New Press Detail Panel

Add a right-side drawer or lower detail area that updates when a press is selected.

Sections:

Machine identity

Show:

- Press/Machine #

- Display name
- robot type(s)
- robot model/controller(s)
- tool numbers
- EOAT types
- physical audit count
- compatible entry count
- last updated

Physical audit records

For each physical audit:

- Audit ID
- date
- status
- EOAT type
- tool
- known issues
- completion profiles
- photos
- open items
- validation count

Buttons:

- Open audit
- Jump to next missing field
- Export audit summary
- Create follow-up

Compatible records

For each compatibility row:

- Audit ID
- source audit ID
- compatibility source
- machine
- tool
- inherited fields
- overridden fields
- stale warning count
- source audit link

Buttons:

- Open compatibility row

- Open source physical audit
- Refresh compatibility from source
- Compare to source

Evidence map

Show counts and links:

- notes
- tags
- open items
- photos
- validation findings
- reports
- documentation gaps
- standards compliance warnings

Standards compliance

Show:

- average compliance score
- worst compliance category
- open standards issues
- missing CAD
- missing BOM
- missing process binder
- missing spare parts
- missing PM checklist

Next recommended action

Generate one or more recommendations:

- Complete physical audit fields.
- Add photo evidence.
- Resolve validation finding.
- Add BOM/CAD/process binder info.
- Refresh stale compatibility rows.
- Mark pilot candidate evidence.
- Run press summary export.

7.3 Backend Expansion

Update `core/press_view.py`.

Add:

```
@dataclass(frozen=True)
class PressDetail:
    group: PressViewGroup
    robot_types: list[str]
    robot_models: list[str]
    evidence_counts: dict[str, int]
    completion_rollup: dict[str, int]
    compatibility_links: list[CompatibilityLink]
    next_actions: list[PressNextAction]
```

Add builder:

```
def build_press_detail(project_root: str | Path, machine: str) -> PressDetail:
    ...
```

7.4 UI Commands

Add Press View commands:

- Open selected machine detail
- Copy machine summary
- Export machine detail report
- Filter machines with physical audit missing
- Filter machines with only compatibility rows
- Filter machines with validation warnings
- Filter machines with missing photos
- Filter machines with pilot candidate

7.5 Tests

Add tests for:

- Press detail builds for machine with physical and compatible rows.
- Source audit links resolve.
- Evidence counts include notes/tags/photos/open items.
- Compatibility rows do not count as physical verification.
- Next recommended action prioritizes blockers.
- Export includes physical and compatible sections.

7.6 Acceptance Criteria

- Selecting a press gives a meaningful detail view.
- “Open Machine Group” no longer only displays a generic message.
- The user can understand what is done and not done for each machine.

- Physical and compatibility data remain clearly separated.
-

8. Phase 5: Upgrade Open Items into a Workflow Board

8.1 Objective

Open Items should become a true work queue, not just a list of generated findings.

8.2 Board Columns

Add board view columns:

- Open
- In Progress
- Waiting on Info
- Blocked
- Fixed at Source
- Dismissed / Overridden

Keep table view as an option.

8.3 Required Item Actions

For each open item:

- Jump to target
- Open source
- Create note
- Create follow-up
- Dismiss with reason
- Copy item summary
- Export selected item
- Show source data
- Show fix recommendation

8.4 Source-Fix Routing

Open items generated from source data should route to where they can be fixed.

Examples:

Validation finding

Route to:

- Workbook Health page
- selected finding
- audit field if available

Missing photo evidence

Route to:

- Photos page
- related audit ID
- required shot checklist

Documentation gap

Route to:

- Standards Docs page
- related audit or machine

Tag assignment

Route to:

- Tags page
- selected assignment

Note follow-up

Route to:

- Notes page
- selected note

Action item

Route to:

- Action Item details or workbook row

8.5 Owners and Due Dates

Add support for:

- owner

- due date
- priority
- blocked reason
- waiting reason
- resolution note

Generated items should still not be falsely marked resolved. Manual state changes should either:

- update the source,
- dismiss with reason,
- or record workflow state without claiming source is fixed.

8.6 Analytics

Add cards:

- total open
- critical open
- overdue
- blocked
- waiting on info
- missing evidence
- data conflicts
- fixed this week
- dismissed/overridden

Add charts/tables:

- open items by machine
- open items by category
- open items by source
- aging distribution
- top repeated fields
- top repeated machines

8.7 Tests

Add tests for:

- Generated items cannot be manually resolved without source fix.
- Dismissal requires reason.
- Board columns group correctly.
- Source-fix routing returns correct target payload.
- Open item summary counts are correct.
- Fixed-at-source history appears in include_resolved mode.

8.8 Acceptance Criteria

- Open Items page can be used as a daily task board.
 - Each item explains how to fix it.
 - Each item can jump to the proper source page.
 - Dismissals are auditable.
 - No generated issue is falsely marked resolved.
-

9. Phase 6: Photo Evidence Manager

9.1 Objective

Make photo handling a full evidence workflow.

9.2 Photo Intake

Add drag/drop or folder picker intake.

User flow:

1. Select incoming photo folder.
2. Choose audit ID or machine.
3. Select evidence type.
4. Preview rename.
5. Confirm intake.
6. Files are copied or moved to project photo folder.
7. Photo Index is updated.
8. Audit photo fields are updated if needed.
9. Missing evidence checklist refreshes.

9.3 Evidence Types

Default evidence types:

- Overall EOAT
- Tool connection
- Vacuum cups
- Grippers
- Cylinders
- Sensor mounting
- Tubing routing
- Quick disconnects
- Cable management
- Robot-side pneumatics

- Wear/damage
- Tool label / ID plate
- Other

9.4 Required Shot Checklist

Create required shot rules based on audit data.

Examples:

- Vacuum EOAT requires vacuum cups.
- Gripper EOAT requires grippers.
- Hybrid requires vacuum cups and grippers.
- Cylinder fields populated require cylinder photo.
- Sensors Present? = Yes requires sensor mounting photo.
- Electrical/Wiring Present? = Yes requires cable management photo.
- Quick Disconnects Present? = Yes requires quick disconnect photo.
- Pneumatic circuit counts populated require pneumatic routing photo.

Suggested file:

- `core/photo_evidence_rules.py`

9.5 Thumbnails

Add optional thumbnail cache:

- `00_Project_Admin/cache/photo_thumbnails/`

Rules:

- Do not store thumbnails in repo.
- Thumbnails are local project artifacts.
- Missing files should show broken evidence status.

9.6 Photo Evidence Score

For each audit:

```
Photo Evidence Coverage:
Overall: present
Tooling: missing
Sensors: present
Pneumatics: missing
```

Cable Management: N/A
Coverage: 58%

9.7 Integration

Add photo coverage to:

- Audit Coach
- Press View
- Open Items
- Workbook Health
- Reports
- Final Handoff

9.8 Tests

Add tests for:

- Required shot rules for Vacuum.
- Required shot rules for Mechanical / Gripper.
- Required shot rules for Hybrid.
- Cylinder photo requirement only when cylinder fields populated.
- Broken photo paths generate missing evidence items.
- Photo index search still works.
- Intake preview does not modify files.
- Confirmed intake writes index rows safely.

9.9 Acceptance Criteria

- User can intake photos without manually renaming everything.
- App can tell which evidence photos are missing.
- Press View can show photo coverage.
- Audit Coach can warn about missing required evidence.
- Photo paths are validated.

10. Phase 7: Evidence-Driven FMEA-Lite

10.1 Objective

FMEA should be generated from actual audit evidence, not manually invented.

10.2 Failure Mode Suggestions

Generate suggestions from:

- Known Issues
- Drop/Mis-Pick History
- Maintenance Frequency
- Cycle Time Concern?
- Scrap/Quality Concern?
- Tubing Condition
- Cable Management Condition
- EOAT Alignment Condition
- Fastener/Locking Hardware Present?
- Sensor fields
- Pneumatic circuit fields
- notes
- tags
- open items
- validation findings

10.3 Suggested Failure Modes

Default failure modes:

- Vacuum loss
- Cup wear or damage
- Mis-pick
- Part drop
- Tubing kink/leak
- Sensor failure
- Electrical/cable management issue
- Mechanical wear
- EOAT misalignment
- Fastener/locking hardware issue
- Quick-disconnect mismatch
- Pneumatic circuit ambiguity
- Cylinder failure
- Changeover error
- Documentation gap causing maintenance risk

10.4 FMEA Data Model

Suggested file:

- `core/fmea_suggestions.py`

Suggested model:

```

@dataclass(frozen=True)
class FmeaSuggestion:
    failure_mode: str
    effect: str
    cause: str
    detection_method: str
    severity: int
    occurrence: int
    detectability: int
    rpn: int
    confidence: str
    evidence: list[FmeaEvidence]
    recommended_action: str
    linked_audits: list[str]
    linked_machines: list[str]

```

10.5 Confidence Labels

Use:

- High: repeated evidence across multiple audits or multiple source types
- Medium: one strong physical audit source
- Low: inferred from missing data or weak text match

10.6 UI

FMEA page should show:

- suggested entries
- accepted entries
- rejected entries
- evidence panel
- severity/occurrence/detectability editor
- RPN calculation
- mitigation action
- linked open item
- export report

10.7 Tests

Add tests for:

- Known issue text maps to failure mode.
- Drop/mis-pick maps to part drop/mis-pick failure mode.
- Tubing condition maps to tubing failure.

- Missing sensor info does not claim sensor failure, only low-confidence risk.
- RPN calculation is correct.
- Accepted suggestions export correctly.

10.8 Acceptance Criteria

- FMEA page can suggest entries from audits.
 - User can accept/edit/reject suggestions.
 - Every suggested entry shows evidence.
 - Confidence is visible.
 - Export includes evidence links.
-

11. Phase 8: Transparent Pilot Candidate Scoring

11.1 Objective

Pilot candidate scoring should be explainable and defensible.

11.2 Inputs

Use:

- Known Issues
- Drop/Mis-Pick History
- Maintenance Frequency
- Cycle Time Concern?
- Scrap/Quality Concern?
- Changeover Difficulty
- Priority
- Pilot Candidate?
- FMEA risk
- open items
- standards compliance
- documentation completeness
- photo evidence
- estimated implementation effort

11.3 Scoring Categories

Default weights:

- Downtime / reliability impact: 30%
- Quality / scrap impact: 25%
- Ease of implementation: 15%

- Safety / maintenance risk: 15%
- Standardization value: 15%

Allow weights to be adjusted in UI.

11.4 Candidate Evidence Matrix

For each candidate:

- score
- rank
- confidence
- affected machines
- issue evidence
- KPI evidence
- missing data
- recommended pilot action
- expected benefit
- implementation blockers

11.5 Sensitivity Analysis

Add:

- "What changed this ranking?"
- "What if ease of implementation matters more?"
- "What data is missing?"
- "Top low-effort wins"
- "Top high-impact candidates"
- "Top risky but valuable candidates"

11.6 Tests

Add tests for:

- Score calculation is deterministic.
- Weight changes affect ranking.
- Missing evidence lowers confidence.
- Pilot Candidate? = Yes boosts but does not override all scoring.
- Export includes score explanation.

11.7 Acceptance Criteria

- Pilot candidate page explains every score.
- User can adjust weights.
- Candidate ranking is reproducible.
- Missing data is visible.

- Export is leadership-friendly.
-

12. Phase 9: KPI Dashboard Truth Labels

12.1 Objective

KPI Dashboard must distinguish actual measured data, audit-observed data, estimated data, and missing data.

12.2 KPI Data Categories

Actual measured

Examples:

- downtime minutes from real downtime source
- scrap count from quality data
- cycle time from measured production data

Audit-observed

Examples:

- drop/mis-pick history text
- maintenance frequency
- cycle time concern
- scrap/quality concern

Estimated / subjective

Examples:

- expected cycle time improvement
- priority
- pilot impact estimate

Missing

Examples:

- no downtime source
- no scrap data
- no before/after baseline
- no pilot result

12.3 UI

Every chart/card should show:

- source type
- date range
- record count
- confidence
- missing-data warning if applicable

Example:

```
Part Drop Risk:  
Source: audit-observed  
Confidence: Medium  
Records: 18 audits  
Missing: 4 audits have unknown drop/mis-pick history
```

12.4 Reports

KPI reports should include a “Data Confidence” section.

12.5 Tests

Add tests for:

- KPI records classify by source type.
- Missing data warnings generated.
- Estimated values not labeled measured.
- Report includes confidence section.

12.6 Acceptance Criteria

- KPI dashboard never pretends estimates are measured values.
 - Missing data is visible.
 - KPI reports are leadership-friendly and truthful.
-

13. Phase 10: EOAT-Type-Specific PM Checklist Generator

13.1 Objective

PM checklists should be generated from EOAT type and actual audit fields.

13.2 Checklist Types

Vacuum EOAT

Include:

- Inspect vacuum cups for wear/damage
- Verify cup mounting
- Inspect vacuum tubing
- Verify vacuum confirmation
- Check vacuum generator
- Inspect pneumatic quick disconnects
- Verify circuit labels/routes

Mechanical / Gripper EOAT

Include:

- Inspect gripper fingers/jaws
- Verify gripper actuation
- Inspect gripper mounting
- Check pressure circuit
- Verify part-present detection if present
- Inspect mechanical wear
- Check fasteners

Hybrid EOAT

Include both vacuum and gripper items.

Cylinder section

If cylinder fields populated:

- Inspect cylinder body/mounting
- Verify cylinder type
- Verify stroke/rotation
- Inspect rod/play

- Inspect fittings
- Check actuation timing
- Check air leaks

Documentation section

Include:

- CAD available
- BOM available
- process binder populated
- spare parts identified
- PM frequency assigned

13.3 Output Formats

Support:

- Markdown
- Excel
- DOCX if current dependency remains acceptable

13.4 Tests

Add tests for:

- Vacuum checklist includes vacuum items.
- Gripper checklist includes gripper items.
- Hybrid includes both.
- Cylinder section appears only when applicable.
- Checklist links back to audit ID.
- Generated file does not overwrite existing file.

13.5 Acceptance Criteria

- PM checklists are specific to EOAT data.
 - Generic checklist behavior still works.
 - Generated checklists are saved safely.
 - Reports clearly state source audit.
-

14. Phase 11: BOM and Spare Parts Standardization Engine

14.1 Objective

BOM / Spare Parts should identify standardization opportunities, not just summarize fields.

14.2 Normalize Known Aliases

Support mappings such as:

- Large Double Gripper → MHZL2-16D
- Small Double Gripper → MHZL2-10S

Add config-driven mapping:

- `data_templates/part_aliases.example.json`
- local ignored config for real mappings if needed

14.3 Analysis Outputs

Show:

- common gripper models
- common cup types
- common sensor models
- common fitting/quick disconnect types
- unknown part entries
- missing part numbers
- repeated aliases
- one-off unusual parts
- machines using each part
- recommended stocking priority

14.4 Standardization Opportunity Types

- High reuse part
- Missing standard part number
- Free-text alias needs normalization
- One-off part should be reviewed
- Part appears in audit but not BOM
- BOM exists but spare parts not identified

14.5 Tests

Add tests for:

- Alias normalization.
- Unknown model detection.
- Frequency counts.
- Standardization recommendations.
- Export includes machines and audits.

14.6 Acceptance Criteria

- BOM page identifies reusable parts.
 - Friendly names are mapped to actual part numbers.
 - Unknown/free-text entries are flagged.
 - Export supports purchasing/inventory conversation.
-

15. Phase 12: Search Upgrade

15.1 Objective

Global search should become ranked, indexed, and explainable.

15.2 SQLite FTS Index

Create local search index under:

- `00_Project_Admin/cache/search_index.sqlite`

Index:

- audits
- audit fields
- machines
- notes
- tags
- open items
- validation findings
- reports
- photos
- press groups

15.3 Search Result Explanation

Each result should include:

- matched source
- matched field
- snippet
- rank score
- why matched

Example:

```
Matched because Known Issues contains "drop"  
Audit: EOAT-2026-014  
Machine: 12  
Field: Known Issues
```

15.4 Saved Searches

Add saved searches:

- Missing photos
- Open critical items
- Hybrid incomplete
- Pilot candidates
- Missing BOM
- Missing CAD
- Needs validation
- Compatibility rows stale
- Follow-ups due this week
- Presses with only compatible entries

15.5 Tests

Add tests for:

- FTS index builds from demo data.
- Search falls back if FTS unavailable.
- Exact audit ID ranks first.
- Exact machine ranks high.
- Open critical items saved search works.
- Missing BOM saved search works.

15.6 Acceptance Criteria

- Search is faster and more relevant.
 - Result explains why it matched.
 - Fallback search still works.
 - No external services required.
-

16. Phase 13: Performance Doctor

16.1 Objective

Performance page should explain why the app is slow and recommend fixes.

16.2 Expand Performance Data

Track:

- startup load config time
- window creation time
- page creation time
- workbook open time
- workbook save time
- validation scan time
- report generation time
- cache read time
- cache write time
- UI render time
- event dispatch time
- background queue wait time
- workbook lock wait time

16.3 Performance Doctor Output

Example:

```
Audit Save took 31.2s.
```

```
Breakdown:
```

- ```
- Workbook load/save: 22.4s
- Compatibility sync: 5.8s
- Event dispatch: 0.3s
- UI render: 1.1s
```

Likely cause:

Workbook save time dominates.

Recommended:

- Skip compatibility sync when no propagated fields changed.
- Save workbook once per operation.
- Use cache invalidation instead of full refresh.

## 16.4 Tests

Add tests for:

- Performance summary identifies slowest operation.
- Recommendations generated for slow workbook save.
- Recommendations generated for cache miss/stale.
- Recommendations generated for validation slowness.

## 16.5 Acceptance Criteria

- Performance page shows not just slow operations, but likely causes.
  - Recommendations are actionable.
  - No private path leaks in exported diagnostics.
- 

# 17. Phase 14: Release Readiness Proof Report

## 17.1 Objective

Release Readiness should prove the repo is safe to publish.

## 17.2 Add Public Repo Safety Report

Generate a markdown report showing:

- staged files scanned
- tracked files scanned
- ignored project root confirmed
- demo data confirmed synthetic
- no real workbook artifacts
- no generated reports/logs/cache/backups
- no capacity files
- no customer/mold/part/private production data
- no internal shared-drive paths
- no photos/screenshots

- local config ignored
- README present
- USAGE docs present if applicable
- tests status
- safety audit status

## 17.3 UI Actions

Add:

- Run staged safety audit
- Run full safety audit
- Generate public safety proof report
- Copy safe commit checklist
- Open offending file
- Suggest `.gitignore` rule
- Open sanitization report

## 17.4 Tests

Add tests for:

- Safety proof report generated.
- Known unsafe demo fixture triggers blocker.
- Ignored local config is not included.
- Report does not contain private path examples except sanitized patterns.

## 17.5 Acceptance Criteria

- Release Readiness can generate a shareable safety proof report.
  - Unsafe files are clearly explained.
  - Commit checklist is copyable.
  - Existing repo safety audit still works.
- 

# 18. Phase 15: Feature Registry Architecture

## 18.1 Objective

Move from a page registry plus scattered commands/search/actions toward a feature registry.

## 18.2 FeatureSpec

Suggested file:

- `app/feature_registry.py`

Suggested model:

```
@dataclass(frozen=True)
class FeatureSpec:
 id: str
 label: str
 page_key: str
 section: str
 description: str
 commands: tuple[str, ...] = ()
 search_sources: tuple[str, ...] = ()
 report_generators: tuple[str, ...] = ()
 event_listeners: tuple[str, ...] = ()
 help_topics: tuple[str, ...] = ()
 data_dependencies: tuple[str, ...] = ()
 modifies_files: bool = False
```

## 18.3 Use Cases

The feature registry should eventually drive:

- sidebar
- command palette
- tool registry
- help page
- onboarding
- release readiness checks
- search providers
- report catalog

## 18.4 Migration Strategy

Do not replace `PAGE_SPECS` immediately.

Instead:

1. Add `FeatureSpec`.
2. Link each feature to existing page key.
3. Generate command metadata from feature registry where possible.

4. Keep `PAGE_SPECS` as the source for actual page loading.
5. Gradually migrate scattered command/search/help definitions.

## 18.5 Tests

Add tests for:

- every feature maps to a valid page key
- every page has a feature or explicit reason
- no duplicate feature IDs
- feature command IDs exist
- help topics load

## 18.6 Acceptance Criteria

- Feature registry exists.
  - It does not break current page loading.
  - At least Audit, Press View, Open Items, Workbook Health, Scheduled Reports, Photos, and Reports are represented.
  - Future features can be added without hardcoding everything in `dashboard_ui.py`.
- 

# 19. Phase 16: In-App Help and Field Dictionary

## 19.1 Objective

The app should explain fields and workflows inside the UI.

## 19.2 Field Dictionary

Create:

- `core/audit/field_dictionary.py`
- optional data template JSON if preferred

For each field:

- field name
- plain-English definition
- example values
- bad/ambiguous values
- required/important/optional
- completion profile relevance
- applicable EOAT types
- default behavior



- whether blank is allowed
- whether Unknown / Not Checked is allowed
- whether it can be compatibility-derived
- whether it appears in reports

## 19.3 UI

Add help drawer or field info popover.

On Audit page:

- each field row can show help
- Audit Coach can link to field help
- Guided Audit mode can show short instructions

## 19.4 Workflow Help Topics

Add help topics:

- How to audit an EOAT
- How compatibility rows work
- How Empty Only view works
- How Audit Coach calculates completion
- How scheduled reports work
- How to fix workbook validation findings
- How to manage photo evidence
- How to prepare final handoff
- How to keep real data out of GitHub

## 19.5 Tests

Add tests for:

- every audit field has dictionary entry or explicit skip reason
- field help loads for common fields
- unknown field returns generic help
- completion profile references valid fields

## 19.6 Acceptance Criteria

- User can understand what each major audit field means.
- Help content is local.
- Help content is not snarky or informal.
- Help content is suitable for workplace use.

## 20. Implementation Order

Recommended implementation order:

### Milestone A: Command and workflow foundation

1. Add command provider infrastructure.
2. Expand command palette commands.
3. Add disabled reasons.
4. Add command tests.

### Milestone B: Audit usability

1. Add Guided Audit Mode shell.
2. Add audit save preview/diff.
3. Add completion profiles.
4. Add cylinder completion behavior if cylinder fields are present.
5. Add audit field help.

### Milestone C: Machine workflow

1. Add Press Detail builder.
2. Add Press Detail UI.
3. Add source/compatibility comparison.
4. Add next recommended actions.

### Milestone D: Work queue and evidence

1. Expand Open Items board.
2. Add source-fix routing.
3. Add Photo Evidence Manager.
4. Add photo checklist and coverage score.

### Milestone E: Engineering analysis

1. Add FMEA suggestions.
2. Add pilot candidate evidence matrix.
3. Add KPI truth labels.
4. Add PM checklist generator expansion.
5. Add BOM standardization engine.

### Milestone F: System polish

1. Add ranked search index.
2. Add Performance Doctor.

3. Add Release Safety Proof Report.
  4. Add Feature Registry.
  5. Add onboarding/help topics.
- 

## 21. Testing Requirements

### 21.1 Run all tests

Every phase must end with:

```
python -m pytest
```

### 21.2 Safety audit

Before committing:

```
python scripts/repo_safety_audit.py --staged
```

Before publishing:

```
python scripts/repo_safety_audit.py --root .
```

### 21.3 UI smoke tests

Where existing UI smoke tests exist, preserve and expand them.

For PySide tests, continue using offscreen rendering if current test strategy does.

### 21.4 No private data

Never commit:

- real audit workbooks
- real photos
- generated reports
- logs
- caches
- backups
- capacity files
- customer names

- mold numbers
  - internal shared-drive paths
  - production-specific private data
  - local config files
- 

## 22. Definition of Done

A phase is done only when:

- Feature works in demo project.
  - Existing workflows still work.
  - Tests are added or updated.
  - No private data is committed.
  - Safety audit passes.
  - UI labels are clear.
  - Background tasks do not freeze the UI.
  - File-writing actions are safe.
  - Workbook-modifying actions use workbook locks where appropriate.
  - User can understand what happened after every action.
  - Reports/logs are saved under the selected project root, not the repo.
  - Generated files do not silently overwrite existing files.
- 

## 23. Final Product Vision

After these expansions, the EOAT Command Center should feel like this:

### Home

Shows what needs attention today.

### Audit

Guides the user through a physical EOAT audit and explains what is missing, what is defaulted, and what will change when saved.

### Press View

Shows the truth about each machine: physical audits, compatibility rows, evidence, issues, validation warnings, photos, and next actions.

## **Open Items**

Acts as a daily task board with source-fix routing.

## **Photos**

Manages visual evidence instead of dumping filenames into a spreadsheet swamp.

## **Workbook Health**

Finds and repairs data issues safely.

## **FMEA**

Suggests failure modes from evidence.

## **Pilot Candidates**

Ranks improvement candidates with explainable scoring.

## **KPI Dashboard**

Clearly separates measured data, observed audit data, estimates, and missing data.

## **PM Checklists**

Generates EOAT-specific maintenance checklists.

## **BOM / Spare Parts**

Finds standardization opportunities and unknown part data.

## **Scheduled Reports**

Runs summaries reliably and explains missed/failed runs.

## **Performance**

Explains why things are slow and what to optimize.

## **Release Readiness**

Proves the repo is safe to publish.

The app should become a local-first EOAT engineering assistant that reduces repeated manual work, catches data problems early, guides the user through audits, and produces credible final deliverables.